MemoryGym(MemoryBench) 分析报告

MemoryGym 现状报告

> 2026-03-09 | Phase 7 完成后

一、核心定位：填补了什么空白？

现有的 LLM 记忆评测（LoCoMo、MemoryAgentBench、LongMemEval）都在测"能不能找到"**——即检索质量。但真实 agent 场景的核心挑战不是检索，而是**"该存什么"。

当一个 agent 面对 600 份文档但只能存 30 条记忆时，它必须做出三个决策：

1. **选择性存储**：哪些信息值得占用有限预算？
2. **记忆维护**：信息更新后能否及时修正？
3. **推理利用**：能否从存储的碎片信息中推导出新结论？

没有任何现有 benchmark 同时测试这三个维度。 MemoryGym 是第一个。

二、真实评测数据：区分力实证

跨模型、跨模板评测结果

模型	参数量	模板	Tier	综合分	广度	维护	推理
Qwen3.5-397B	397B MoE	hospital	lite	90%	100%	100%	50%
Qwen3.5-397B	397B MoE	research	lite	70%	100%	33%	50%

Qwen3.5-397B	397B MoE	city	lite	60%	100%	33%	0%
Kimi-K2.5	--	company	lite	50%	75%	33%	0%
Kimi-K2.5	--	company	lite	40%	50%	0%	33%
MiniMax-M2.5	--	company	lite	50%	50%	67%	0%
Qwen3-32B	32B	company	std	45%	56%	0%	33%
Qwen3-235B	235B MoE	company	lite	30%	20%	100%	0%
DeepSeek-V3	671B MoE	company	hard	37%	20%	0%	61%
Qwen3-14B	14B	company	lite	20%	0%	33%	0%
GPT-OSS-120B	120B	company	std	0%	0%	0%	0%

这些数据证明了什么？

1. **真正的区分力**：从 0% 到 90%，分数跨度巨大。不是所有模型都能拿高分——评测在测真实能力，不是走形式。

2. **维护轴是真正的分水岭**：

- Qwen3.5-397B 在 hospital 拿到 maintenance=100%，在 research 只有 33%

  - **大多数模型 maintenance=0%**——它们根本没有更新修正后的信息

  - 这暴露了当前 LLM 的一个系统性弱点：**缺乏主动记忆维护能力**

3. **模型规模不等于能力**：

- Qwen3-235B (235B) 平均 25%, 低于 Kimi-K2.5 的 50%
- GPT-OSS-120B (120B) = 0%（工具格式不兼容）
- 记忆管理能力与模型规模不完全相关，更依赖架构和训练方式

4. **跨模板一致性**：同一模型（Qwen3.5-397B）在不同领域得分 60-90%，变化来自领域特性而非评测系统偏差。

轨迹深度分析：Kimi-K2.5 company seed=42（40%）

完整的 per-turn 轨迹记录了模型在每个事件中的每一步工具调用和返回结果。以下是一次真实评测的完整行为链：

存储阶段 — 预算分配失策

Code block

```
[1/17] INGEST 10 entities → 10x memory_store + memory_list
       预算: 15 → 5（一口气全存，消耗 10/15 预算）
[2/17] INGEST 10 entities → 2x memory_store
       预算: 5 → 3（只能再存 2 个: Nimbus Aerospace, Cobalt Holdings）
```

模型在第一批全量存储（10/10），导致后续批次预算不足。30 个实体只存了 12 个（40%）。

Correction 阶段 — 操作正确但内容错误

Code block

```
[4/17] CORRECT Stratos Labs.employees
  Turn 0: memory_search("Stratos Labs") → 找到旧记录
  Turn 1: memory_forget(旧ID) → 删除旧记忆
  Turn 2: memory_store("...Employees: 89,132...") → 存入"更新"记忆
  问题: 存入的员工数仍是原始值 89,132，没有替换为修正后的新值!
```

模型理解了"需要更新"，正确执行了 search-forget-store 三步操作，但在生成新内容时直接复制了搜索结果原文，未将修正值替换进去。这是 maintenance 轴为 0% 的根因。

答题阶段 — 10 个问题逐题分析

```
Code block
[8]  synthesis（Debt ratio最高）→ WRONG  搜了5次但选错公司
[9]  retrieval（Cobalt 市值）   → OK     第2批存了，精确匹配 $459,697.6M
[10] update（Stratos Labs 员工）→ WRONG  回答210（correction未生效）
[11] update（Vector Analytics）→ WRONG  第3批没存，无数据
[12] retrieval（Lumen R&D）    → OK     第1批存了，15.83%
[13] retrieval（Titan Networks）→ WRONG  第3批没存
[14] delta（员工变化量）       → WRONG  空答案，229秒超时
[15] retrieval（Stratos Group）→ WRONG  第2批没存
[16] abstention（Atlas Aero）  → OK     正确识别"不知道"
[17] comparison（Cobalt vs Vortex）→ OK  两个都存了，精确对比
```

轨迹揭示的系统性问题

1. 预算感知不足：第一批全存导致后续无法存储，18 个实体被遗漏
2. Correction 形式正确但语义失败：操作流程对（search-forget-store），但存入内容未更新
3. 存储覆盖直接决定 retrieval 得分：存了的实体能精确回答，没存的只能说"不知道"

三、具体原理：评测是怎么工作的？

一次完整评测的流程

```
Code block
seed=42 → CompanyWorld → 生成 30 家公司实体
                         ↓
          渲染为自然语言商业文档（每个约 250 字）
                         ↓
      Agent 在流式事件中处理（预算 15 次写入）：
      ├── [Event 1-3] INGEST: 10 家公司的文档 → 存储决策
```

```
    ├── [Event 4]   QUESTION: "Vector Analytics 有多少办公室？"
    ├── [Event 5-7] INGEST: 10 家公司 → 继续存储
    ├── [Event 8]   CORRECTION: "Vortex Labs 员工数从 5564 修正为 3870"
    ├── [Event 9]   QUESTION: "Vortex Labs 员工数是多少？"（答案=3870）
    ├── [Event 10]  INGEST（隐式矛盾）: Pinnacle 市值悄然变更
    └── [Event 11-20] 更多问题：聚合、推理、弃权...
```

世界模板示例

一个真实生成的实体：

Code block

```
Entity: Nexus Digital (category: Technology)
  revenue_m: 3847        employees: 12453
  market_cap_m: 54245    debt_ratio: 0.73
  offices: 28            profit_margin: 0.156
  rd_budget_m: 892       founded_year: 1987
```

Agent 看到的文档：

Code block

```
QUARTERLY EARNINGS DISCLOSURE -- Nexus Digital

Nexus Digital, a Technology sector leader established in 1987,
maintains 28 offices worldwide. The company's profit margin
stands at 15.6%, achieved with a workforce of 12,453 employees
and R&D investment of $892M...
```

6 个领域模板

模板	领域	实体类型	关系类型
company	商业	公司	

			supplies_to, competes_with
research	学术	研究机构	collaborates_with, advised_by
city	城市	城市	trade_partner, sister_city
hospital	医疗	医院	refers_to, affiliated_with
sport	体育	运动队	rivalry, trades_with
movie	影视	电影	sequel_of, same_director

每个模板：600 个名字 × 10 属性 × 10-12 类别 = 事实上无限的组合空间。

14 种问题类型覆盖真实推理场景

类型	示例	测什么
retrieval	Nexus Digital 有多少员工?	基础存储
update	Vortex Labs 员工数是多少?（修正后）	记忆维护
synthesis	Technology 类中收入最高的是?	排序推理
aggregation	Healthcare 类平均利润率?	聚合计算
cross_category	市值 top-3 的平均负债率?	跨类聚合
ratio	Nexus Digital 人均收入?	派生计算
comparison	A 和 B 谁利润率更高?	比较推理
multi_hop	Technology 中员工最多的公司的收入?	多跳推理
delta	修正前后变化了多少?	变更感知

outlier	哪家公司负债率异常偏离均值?	离群检测
relationship_hop	A 的供应商的收入是多少?	关系图推理
relationship_chain	A-B-C 的供应链末端市值?	链式推理
abstention	对未存储实体提问	识别未知的能力
trick_retrieval	对未存储实体问已知属性	防御盲猜

隐式矛盾（Phase 5 新增）

不同于明确标记的 "CORRECTION NOTICE"，隐式矛盾以普通文档形式出现，但包含与先前信息冲突的值：

Code block

```
原始文档:
Pinnacle Sciences market_cap: 54,245

后来的普通文档（无 CORRECTION 标签）:
Pinnacle Sciences market_cap: 27,599
```

Agent 必须**自行发现**这个矛盾并更新记忆。这模拟了真实场景中信息不一致的挑战。

4 轴评分系统

轴	权重	测什么	计算方式
breadth	0.25	存储广度	retrieval 正确率
maintenance	0.25	记忆维护	update 正确率 x coverage gate
reasoning	0.30	推理能力	

			14 种 comprehension 题型正确率
efficiency	0.20	效率	correct/writes_used

abstention_diagnostic 单独报告，不计入 composite。

四、反作弊：为什么分数不可伪造？

每次变更评分逻辑后，8 种模拟策略必须全部满足不变量：

Code block

```
perfect (全存全更新):          100%  ← 证明题目可答
strategic (70% 存 + 更新):     65%  ← 证明策略有用
naive (40% 存 + 不更新):       19%  ← 证明不更新很致命
guesser (不存 + 猜答案):        0%  ← 证明猜不了
smart_guesser (用中位数猜):     0%  ← 证明巧猜也不行
abstainer (全说不知道):        15%  ← 证明弃权有上限
```

被验证为死路的攻击策略

攻击	为什么失败
背题库	每个 seed 生成完全不同的世界
猜中位数/常见值	整数精确匹配，猜中概率低于 1/1000
只存名字不存值	detect_stored_entities 要求名字+数值同时出现
全说不知道	trick_retrieval 问题有真实答案，弃权=0分

选择性存储操纵问题	问题生成与存储决策独立
eval_salt	同一 seed 不同 salt 产生不同数值，RL 训练无法过拟合

五、前沿对标：学术坐标

与现有 benchmark 的本质差异

维度	MemoryGym	LoCoMo	MemoryAgentBench	LongMemEval
核心测试	存储决策	对话记忆	多轮记忆	长期记忆
预算压力	写入受限	无	无	无
记忆维护	修正+矛盾	无	无	无
可训练	RL 环境	纯评测	纯评测	纯评测
确定性	seed 完全确定	非确定	非确定	非确定
反作弊验证	8 策略不变量	无	无	无

MemoryGym 不是在同一维度上做得更好，而是在测一个全新的维度。

与前沿 RL 训练研究的衔接

MemoryGym 的 RL 训练架构直接对接了 2025-2026 最前沿的 agent RL 范式：

概念	来源框架	MemoryGym 实现
Token Masking	Search-R1, VerlTool	verl_adapter response_mask

Outcome-Based Reward	Agent-R1, Search-R1	get_verifiable_reward()
Multi-Turn RL	VerlTool (GRPO-ARLT)	AgentLoopBase 多轮交互
Curriculum Learning	AgentGym-RL (ICLR 26)	lite/standard/hard 渐进训练
Dual Framework	verl + Megatron	verl_adapter + slime_adapter

参考文献

- DeepSeek-R1 (2501.12948), Agent-R1 (2511.14460), WebAgent-R1 (2505.16421)
- REDSearcher (2602.14234), Search-R1 (2503.09516), VerlTool (2509.01055)
- AgentGym-RL (ICLR 2026), Simia (2511.01824)
- A-Mem (2502.12110), MemGPT/Letta (2310.08560), mem0 (2504.19413)
- LoCoMo (2402.17753), MemoryAgentBench (ICLR 2026), LongMemEval (2024)

六、项目成熟度

Code block

```
代码: 249 tests pass, 0 TODO/FIXME, 所有文件 ≤802 行
模板: 6 个领域 (company/research/city/hospital/sport/movie)
      每个 600 个名字 × 10 属性 × 10-12 类别
评测: 8 个模型, 4 个厂商, 17 次真实评测 (含完整轨迹)
训练: MemoryEnv + verl/slime 适配器 + curriculum 配置
反作弊: 8 种策略, eval_salt, 67+ 不变量检查
轨迹: per-turn tool_calls + tool_results 完整记录
```

评测 Tier 设计

Tier	实体	问题	修正	预算	难度来源

lite	30	10	3	15	基础能力验证
standard	60	20	5	30	信息过载
hard	120	40	10	30	同预算 4x 实体

七、结论

MemoryGym 的价值不在于"又一个 benchmark"，而在于它提出并回答了一个此前没有被系统测试的问题：

> **LLM 在资源受限条件下，能否做出智能的记忆管理决策？**

真实评测数据已经给出了答案：**当前最强模型也只有 60-90%，大多数模型在记忆维护上接近 0%。**这是一个真实存在的能力缺口，而 MemoryGym 是目前唯一能量化这个缺口、并提供训练闭环来缩小它的系统。

阻塞项

- GPU 端到端 RL 训练验证（需 4+ GPU）
- Movie 模板 real eval（需 API key）
- 更多模型 × 更多模板的交叉评测数据